

Proof-to-Byte: Assumption-Minimal, Cross-Substrate Binding Assurance for PQ/T Hybrid KEMs

Ziang Li

Abstract—Post-quantum hybrid key-encapsulation (PQ/T KEMs, combining a lattice KEM with an elliptic-curve KEM) is being deployed across TLS, Signal, and MLS faster than its *deployment safety* can be assured: the same period has seen side-channel breaks (KyberSlash), binding / re-encapsulation breaks (PQXDH; the ML-KEM malicious-key binding failures of Schmieg), and a widening gap between what a hybrid’s *specification* guarantees and what its compiled, multi-platform *implementation* actually does. We present Q-Periapt, an assurance suite for PQ/T hybrid KEMs organised around one principle: *a security property is only as trustworthy as its weakest realization*. Its combiner, ContextBound, achieves binding (MAL-BIND-K-CT and MAL-BIND-K-PK) reducing to collision-resistance of SHA3 *alone*—with no binding assumption on the component KEMs—and we machine-check this in EasyCrypt: the injective encoding is a proved lemma (not an axiom) and the KEM-aware game is the full Cremers–Dax–Medinger Figure 6 game for both implicit and explicit rejection. Crucially, the *same* proven combiner is realized byte-identically across five language faces, three operating systems, and five instruction-set architectures, verified by a six-method conformance matrix and a CI-gated, self-validating source→binary constant-time probe that we show *discriminates* a clean primitive (libcrux ML-KEM: zero secret-dependent branches after compilation) from a leaky one (HQC’s PQClean constant-weight sampler). We integrate the combiner into a production TLS 1.3 path (a rustls CryptoProvider) and evaluate handshake tail latency under real `tc netem`. We are explicit about what is and is not novel: the *binding fact* is a known consequence of recent results; our contribution is the *conjunction*—an assumption-minimal, machine-checked binding result whose proven object is demonstrably identical across a heterogeneous deployment surface, with reproducible gates that catch the failure classes that have broken deployed hybrids.

Index Terms—Post-quantum cryptography, hybrid key encapsulation, binding/committing security, formal verification, constant-time, cross-platform assurance, TLS.

I. INTRODUCTION

THE migration to post-quantum (PQ) key establishment is being done conservatively, through *hybrids*: a PQ KEM (ML-KEM [1]) run alongside a classical KEM (X25519), with the two shared secrets combined so that the session key is secure if *either* component holds. Hybrid KEMs are now shipping in TLS 1.3 (X25519MLKEM768), in Signal’s PQXDH, and in MLS, and are specified in CFRG drafts such as X-Wing [5].

The gap this paper addresses: Hybrids are being deployed faster than their *deployment safety* can be assured. In the same window, the community has seen (i) *side-channel* breaks of PQ implementations (KyberSlash [6]); (ii) *binding* failures—the PQXDH re-encapsulation issue, and Schmieg’s proof that

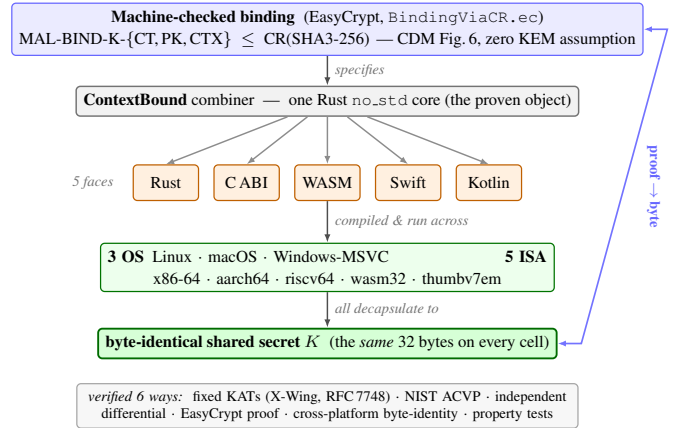


Fig. 1. Proof-to-byte. One machine-checked binding result (top) is realized by a single Rust `no_std` core, exposed through five language faces and run across three operating systems and five instruction-set architectures, all converging on a byte-identical shared secret K (bottom); verified six ways.

ML-KEM in its FIPS-203 *expanded* decapsulation-key form is neither MAL-BIND-K-CT nor MAL-BIND-K-PK [3]; and (iii) a widening gap between a hybrid’s *specification*, its formally-verified source, and the *compiled artifact* that actually runs—across the many languages, operating systems, and instruction sets that real deployments target. This safety question is distinct from, and orthogonal to, *auditability* and *crypto-agility* (which describe *what* crypto is running): it asks whether the running hybrid is actually *safe*.

Approach: We present Q-Periapt, an assurance suite for PQ/T hybrid KEMs built around one principle: a security property is only as trustworthy as its weakest realization. Figure 1 is the organising idea—a single machine-checked binding result whose proven object is realized byte-identically across the whole deployment surface (“proof-to-byte”), guarded by reproducible CI gates that target the specific failure classes above.

Contributions:

- **C1 (Section III).** An *assumption-minimal, machine-checked* binding result: ContextBound reduces MAL-BIND-K-CT, -K-PK, and a context extension -K-CTX to collision-resistance of SHA3 *alone*, with the injective encoding proved and the full CDM Figure-6 game (implicit and explicit rejection) discharged in EasyCrypt.
- **C2 (Section IV).** *Proof-to-byte cross-substrate equivalence:* the same proven combiner is byte-identical across 5 language faces, 3 OSes, and 5 ISAs, verified by a six-

method conformance matrix.

- **C3 (Section V).** *CI-gated assurance* that targets the deployment failure classes: a self-validating source→binary constant-time probe shown to discriminate clean (ML-KEM) from leaky (HQC); an executable demonstration that a lean combiner’s binding is contingent on the component KEM’s key serialization; and two independent symbolic provers.
- **C4 (Section VI).** A production TLS 1.3 path (rustls CryptoProvider), evaluated under real `tc netem`.

What is and is not novel: We state the boundary plainly. *Not novel, and attributed:* that a “hash-everything” combiner is binding from collision-resistance alone while a lean combiner inherits the omitted component’s binding is the Chempat result [4]; ContextBound *is* that construction, not a new primitive. That ML-KEM’s expanded-dk form breaks binding (and the seed form fixes it) is Schmieg’s [3]. The binding-notion lattice is CDM’s [2]. That a constant-time harness must mark only secret sub-fields is standard practice [6]. *Novel:* the conjunction $C1 \wedge C2$ —a CR-only, machine-checked binding result whose proven object is demonstrably byte-identical across a heterogeneous substrate, which no prior work provides; the self-validating source→binary *discriminator* as a reusable CI artifact; and the operationalized design invariant that binding the ciphertext and public key in the KDF makes hybrid binding robust to the component KEM’s key-serialization format. We do **not** claim stronger binding than X-Wing (Section III), nor any live exploited deployment.

II. BACKGROUND AND THREAT MODEL

A. PQ/T hybrid KEMs and combiners

A hybrid KEM combines component shared secrets ss_{pq} and ss_{tr} into a session key K . The design space ranges from simple *concatenation* KDFs (X25519MLKEM768) to combiners that additionally absorb ciphertexts and public keys. X-Wing [5] uses a fixed “lean” combiner $K = \text{SHA3}(ss_{pq} \parallel ss_{tr} \parallel ct_{tr} \parallel pk_{tr} \parallel \text{label})$, omitting the ML-KEM ciphertext and public key.

B. KEM binding

We use the Cremers–Dax–Medinger (CDM) X-BIND- P - Q framework [2]: an adversary of class $\text{MAL} \supseteq \text{LEAK} \supseteq \text{HON}$ wins if two transcripts *agree* on quantity P and *disagree* on quantity Q , with $P, Q \in \{K, \text{PK}, \text{CT}\}$. The MAL adversary supplies its own (possibly inconsistent) key material. For implicitly-rejecting KEMs such as ML-KEM, MAL-BIND-K-CT and MAL-BIND-K-PK are the achievable ceiling on those axes; the dual X-BIND-CT-* notions (a ciphertext binding the key or public key) are *structurally unachievable*, since decapsulation never returns $\perp$.

C. Threat model

We consider three adversaries. The *malicious-key (MAL)* binding adversary supplies adversarial keypairs and seeks colliding transcripts (Section III). The *binary-level constant-time*

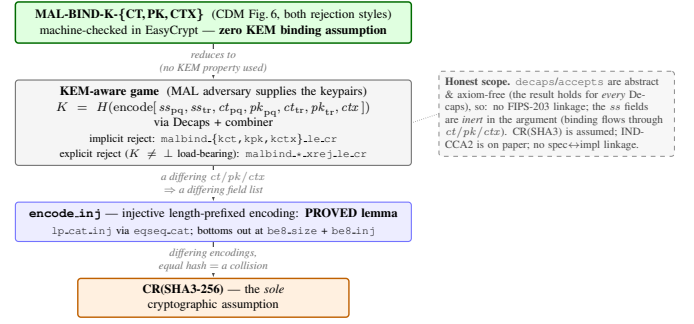


Fig. 2. The kernel as a reduction tower: MAL-BIND-K-{CT,PK,CTX} (CDM Figure 6, both rejection styles) reduces—using no property of Decaps—through the proved `encode_inj` to collision-resistance of SHA3, the sole cryptographic assumption.

adversary observes secret-dependent control flow / memory indexing in the *compiled* code (Section V). The *downgrade/agility* adversary attacks profile and policy negotiation (handled by fail-closed policy binding; out of scope here). Confidentiality of the component primitives (IND-CCA2 of ML-KEM, the Diffie–Hellman assumption for X25519) is assumed, not reproved.

III. CONTEXTBOUND AND THE MACHINE-CHECKED KERNEL

A. Construction

ContextBound derives $K = \text{SHA3-256}(\text{encode}[\text{LABEL}, \text{suite_id}, \text{policy_version}, ss_{pq}, ss_{tr}, ct_{pq}, pk_{tr}])$ where `encode` concatenates each field under a fixed-width 8-byte big-endian length prefix. Two profiles are shipped: `CompatXWing` (byte-exact X-Wing, for interoperability) and `ContextBound` (the hash-everything profile above). The suite is open source.¹

B. The EasyCrypt development

Figure 2 shows the mechanized reduction. The encoding’s injectivity, `encode_inj`, is a *proved lemma*: it reduces, via self-delimiting length-prefixed fields, to two elementary facts about an 8-byte big-endian length field. On top of it we discharge (i) a generic transcript-collision reduction `bind_le_cr`; and (ii) the *KEM-aware* game, in which the MAL adversary supplies the keypairs and K is *derived* via the component Decaps functions and the combiner. We mechanize both the implicit-rejection specialization (`malbind.*_le_cr`) and the fully general CDM Figure-6 game with explicit rejection (`malbind.*_xrej_le_cr`), where the $K \neq \perp$ side condition is *present and load-bearing*—removing it makes the proof fail. Every theorem reduces only to CR(SHA3); the development compiles under the EasyCrypt checker, and each theorem is verified load-bearing on `encode_inj` by an adversarial negative control.

¹<https://github.com/billlza/q-periapt>

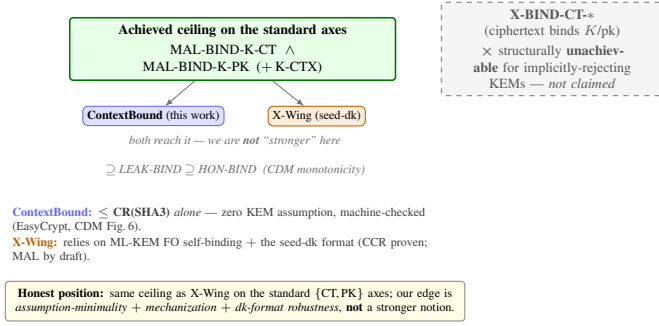


Fig. 3. Honest binding position. Both schemes reach the standard ceiling; our edge is assumption-minimality, mechanization, and dk-format robustness—not a stronger notion. X-BIND-CT* is structurally unachievable for implicitly-rejecting KEMs and is not claimed.

C. Honest binding position

Figure 3 states our position precisely. On the standard $\{CT, PK\}$ axes, ContextBound and a correctly-implemented seed-dk X-Wing both reach the MAL-BIND-K- $\{CT, PK\}$ ceiling; we are *not* “stronger” there. Our edge is (a) *assumption-minimality*: ContextBound’s binding reduces to CR(SHA3) alone, whereas X-Wing’s relies on ML-KEM’s Fujisaki–Okamoto self-binding together with the seed-dk format; (b) *mechanization*; and (c) *dk-format robustness* (Section V). Honest scope of the mechanization: Decaps is modelled as an abstract function (which is exactly why “zero KEM assumption” is literal), so there is no FIPS-203 linkage and the shared-secret fields are inert in the binding argument; CR(SHA3) is assumed; IND-CCA2 robustness is argued on paper; and there is no specification-to-implementation linkage proof.

IV. PROOF-TO-BYTE CROSS-SUBSTRATE REALIZATION

A single dependency-free Rust `no_std` core implements the combiner and the binding/encoding logic; vetted backends (libcrux ML-KEM, x25519-dalek, libcrux SHA3) are injected through traits. The core is exposed through five language faces (Rust, a C ABI, WebAssembly, Swift, Kotlin). Because the proven object is one piece of code, the question becomes whether *every* realization produces the same bytes.

Table I lists the six orthogonal methods that answer this. They are deliberately heterogeneous in their oracle and in what they catch, so a defect that escapes one is caught by another. Table II reports the substrate coverage with honest per-cell status: the byte-identical shared secret is *run* natively on x86-64, aarch64, and wasm32; on riscv64 and the bare-metal thumbv7em target the core cross-compiles cleanly and byte-identity follows by construction from the FIPS-deterministic encodings and identical source, but is not executed on a native runner. This honest accounting is what makes the cross-substrate claim defensible.

V. CI-GATED ASSURANCE

A. A source→binary constant-time discriminator

libcrux proves its ML-KEM secret-independent at the *source* level (a typed discipline, machine-checked). The orthogonal

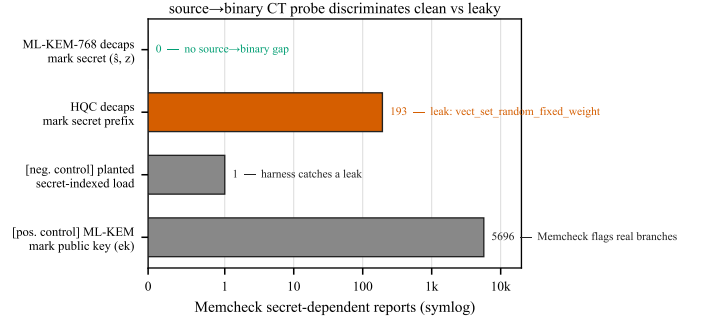


Fig. 4. The source→binary CT probe discriminates: the genuine ML-KEM secret yields zero flags; the same harness flags HQC’s constant-weight sampler (193) and validates itself with two controls.

binary-level question—does the compiler reintroduce a secret-dependent branch on the genuine secret path?—is answered by a probe that marks only the genuinely-secret decapsulation-key sub-fields $(\hat{s}, z)$ and runs the real libcrux decapsulate under Memcheck. The probe is *self-validating*: a planted secret-indexed load is caught (harness sanity), and marking the embedded *public* key flags 5696 reports (Memcheck demonstrably sees real branches in this binary). With only the genuine secret marked, the probe reports **zero** flags on aarch64: source-level secret-independence survives compilation. Run against the suite’s unaudited HQC backend, the same probe flags **193** reports localized to `vect_set_random_fixed_weight`—so the probe is a *discriminator* (Fig. 4): clean ML-KEM versus leaky HQC in one framework. (HQC’s leak is known; the contribution is the discriminating, CI-gated artifact and the corollary that per-backend constant-time gating is necessary, not decorative.)

B. Binding is contingent on the component dk format

We demonstrate, executably against real libcrux, that a lean (X-Wing-shaped) combiner’s MAL-BIND-K-PK is *contingent* on the component KEM’s key-serialization format. Using Schmieg’s expanded-dk construction, two distinct ML-KEM public keys yield the same shared secret; the lean combiner—which omits pk_{pq} —then collides, whereas ContextBound, which absorbs pk_{pq} , does not. We stress that this is *not* a break of X-Wing, which mandates the seed-dk format that defeats the attack; it is a robustness separation of the combiner *shape*, and the operationalized lesson that binding ct/pk in the KDF removes a latent dependence on the component library’s serialization choice.

C. Multi-prover assurance

Beyond the computational EasyCrypt proof, the server-authenticated hybrid handshake is modelled in two independent symbolic provers, Tamarin and ProVerif, each establishing that the session key survives the compromise of *either* component KEM. All three formal artifacts are CI-gated.

VI. PRODUCTION INTEGRATION AND EVALUATION

We expose ContextBound (and CompatXWing) as a TLS 1.3 key-exchange group via a rustls `CryptoProvider`, using

TABLE I
THE SIX ORTHOGONAL VERIFICATION METHODS.

Method	Reference / oracle	Independence	What it catches
Fixed KATs	X-Wing draft vectors; RFC 7748	published vectors	combiner and X25519 spec-conformance regressions
NIST ACVP	<i>usnistgov</i> ACVP, full FIPS family	NIST ground truth	primitive non-conformance (FIPS 203/204/205)
Multi-backend differential	RustCrypto ml-kem/ml-dsa; orion X25519	independent re-implementations	implementation bugs (output must stay byte-identical)
EasyCrypt proof	machine-checked; CR(SHA3)	formal (computational)	binding-design flaws in the combiner
Cross-platform byte-identity	shared test vector	5 faces $\times$ substrates	per-substrate divergence (same K everywhere)
Generative property tests	proptest: 7 properties $\times$ 128 cases	randomized	edge-case violations (injectivity, domain separation, K-CTX)

TABLE II
CROSS-SUBSTRATE COVERAGE ($\checkmark$ VERIFIED, CI OR LOCAL).

(a) ISA targets (*Rust core*; the byte-level claim)

ISA	runner	build	byte-id. K	binary-CT
x86-64	Linux, Win-MSVC	$\checkmark$	$\checkmark$ (run)	$\checkmark$ Memcheck
aarch64	Linux, macOS	$\checkmark$	$\checkmark$ (run)	$\checkmark$ Memcheck
wasm32	node	$\checkmark$	$\checkmark$ (run)	source-CT
riscv64	Linux (cross)	$\checkmark$	by constr. [†]	source-CT
thumbv7em	bare-metal	$\checkmark$	by constr. [†]	n/a

[†] cross-compiled, not run natively; identity follows from FIPS-deterministic encodings + identical source.

(b) language face $\times$ OS (shared vector $\rightarrow$ same K)

Face	Linux	macOS	Windows
Rust	$\checkmark$	$\checkmark$	$\checkmark$
C ABI	$\checkmark$	$\checkmark$	$\checkmark$ (MSVC)
WASM (node)	$\checkmark$	$\checkmark$	$\checkmark$
Swift	—	$\checkmark$	—
Kotlin (JVM/FFM)	$\checkmark$	$\checkmark$	—

private-use group codes; a real loopback TLS 1.3 handshake completes over this provider.

We evaluate handshake *time-to-session* (TCP connect plus full TLS 1.3 handshake) over a real loopback socket shaped by `tc netem`, comparing a classical X25519 baseline against the two PQ/T profiles. Figure 5 reports the result: the three curves coincide because latency is RTT-dominated, and the PQ/T overhead is a *fixed* cost ($\sim 190 \mu\text{s}$ of ML-KEM CPU plus the wire bytes of Fig. 6). At realistic RTT this is negligible: at $\text{RTT} \geq 20 \text{ ms}$ the measured overhead is *within run-to-run noise*—its sign varies across repeated runs (Fig. 5b, error bars span two repetitions)—while the only reproducible cost is the $\sim 190 \mu\text{s}$ of CPU visible at RTT 0. The hybrid’s +2.27 KB (one ML-KEM-768 keyshare each way) fits within the existing TLS flights and triggers no additional round-trip. The two combiner profiles are indistinguishable: the stronger binding of the hash-everything combiner is free in latency. We report these measurements honestly as obtained on a virtualized host; a turnkey script reproduces the protocol on quiesced bare-metal hardware for the camera-ready (the RTT-dominated points are already stable across repetitions).

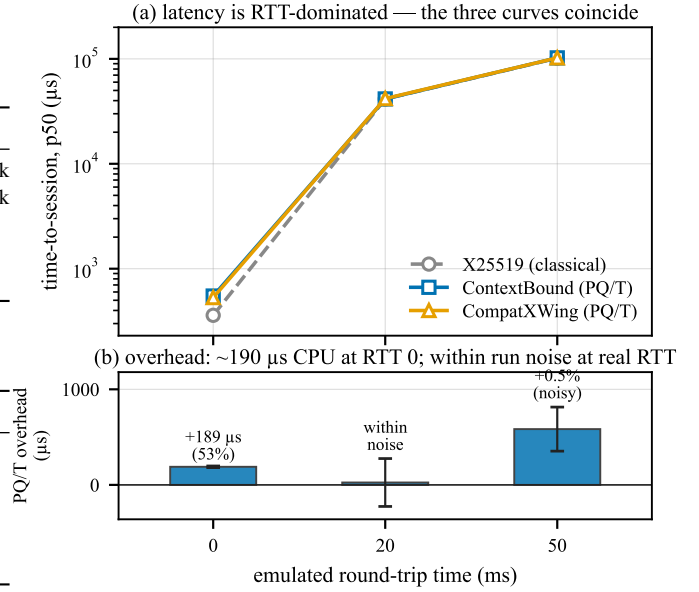


Fig. 5. Time-to-session under real `tc netem` (mean of two repetitions). (a) latency is RTT-dominated—the three curves coincide. (b) the overhead is $\sim 190 \mu\text{s}$ of CPU at RTT 0; at realistic RTT it is within run-to-run noise (error bars span the two repetitions and cross zero at RTT 20 ms).

VII. RELATED WORK

KEM binding. CDM [2] introduced the X-BIND- P - Q lattice and monotonicity; Schmieg [3] proved ML-KEM’s expanded-dk binding failures; Chempat [4] established that hash-everything combiners are binding from collision-resistance while lean combiners inherit the omitted component’s binding (ContextBound is an instance). Context binding is the KEM-layer analogue of context-committing AEAD [7]; the role of public contexts in hybrid KEMs is analysed in [8]. **Side channels.** KyberSlash [6] and the ctgrind/TIMECOP discipline motivate our binary-CT gate; libcrux/HACL* provide source-level guarantees we corroborate and discriminate against. **Hybrids in deployment.** X-Wing [5] and the TLS X25519MLKEM768 group are the deployed baselines we position against. **Formal PQ.** formosa-mlkem mechanizes ML-KEM IND-CCA; the SandboxAQ EasyCrypt-KEMs library mechanizes LEAK-BIND-K-PK; our development adds the MAL Figure-6 game for the hybrid combiner.

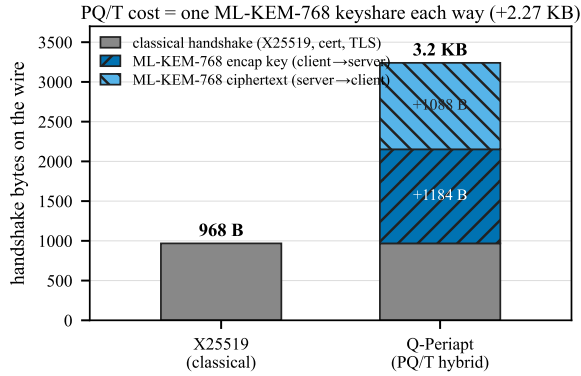


Fig. 6. Handshake wire budget. The PQ/T cost is one ML-KEM-768 keyshare each way (+2.27 KB), which fits inside the existing flights.

VIII. LIMITATIONS

The mechanized binding result is over an *abstract* Decaps: it is exactly the “zero KEM assumption” that makes the result general, but it carries no FIPS-203 linkage, the shared-secret fields are inert in the argument, and CR(SHA3) and IND-CCA2 remain assumptions argued respectively in the model and on paper; there is no specification-to-implementation linkage proof. Binary-level constant-time tooling is mature only on x86-64 and aarch64; riscv64 and wasm32 are covered at source level plus upstream attestation. ACVP byte-identity is conformance, not CMVP certification. The suite is research-grade and has not had an independent third-party audit. The netem evaluation was obtained on a virtualized host; bare-metal reproduction is provided as a script.

IX. CONCLUSION

A PQ/T hybrid is only as safe as its weakest realization. We closed that gap with an assumption-minimal, machine-checked binding result whose proven object is byte-identical across a heterogeneous deployment surface, guarded by reproducible CI gates—a source→binary constant-time discriminator, a binding key-format coupling demonstration, and multi-prover symbolic models—and validated on a production TLS 1.3 path. The genuinely new element is the conjunction, not any single fact; we have been explicit about the boundary throughout.

REFERENCES

- [1] National Institute of Standards and Technology, “FIPS 203: Module-Lattice-Based Key-Encapsulation Mechanism Standard,” 2024.
- [2] C. Cremers, A. Dax, and N. Medinger, “Keeping Up with the KEMs: Binding Security of Key Encapsulation Mechanisms,” *ACM CCS*, 2024. (ePrint 2023/1933.)
- [3] S. Schmieg, “Unbindable Kemmy Schmidt: ML-KEM is neither MAL-BIND-K-CT nor MAL-BIND-K-PK,” *IACR ePrint 2024/523*, 2024.
- [4] J. Güneysu, K. Hövelmanns, *et al.*, “Binding Security of Combined KEMs / Chempat,” *IACR ePrint 2025/1416*, 2025.
- [5] D. Connolly, P. Schwabe, and B. Westerbaan, “X-Wing: The Hybrid KEM You’ve Been Looking For,” draft-connolly-cfrg-xwing-kem (CFRG), 2024.
- [6] D. J. Bernstein, K. Bhargavan, *et al.*, “KyberSlash: Exploiting Secret-Dependent Division Timings in Kyber Implementations,” *IACR TCHES*, 2025.
- [7] M. Bellare and V. T. Hoang, “Efficient Schemes for Committing Authenticated Encryption,” *EUROCRYPT*, 2022.
- [8] “On the Role of Public Contexts in Hybrid KEMs,” *IACR ePrint 2026/140*.